Student Information Dictionary - Project Report

Veda Technology - AI & ML Internship | Level 1, Task 7

Submitted by: J Sanjay, AI & Data Science

1. Introduction

Structured data is the base of every data science workflow. This project builds a small student record system in Python using a dictionary, and implements three core operations on it: search, add and update. The goal is to practice dictionaries and structured data representation.

2. Objective

- Create a Python dictionary that holds student information.
- Implement search, add and update operations as reusable functions.
- Use unique student IDs to identify records.
- Show example outputs for normal and edge cases.

3. Tools Used

- Python 3
- Jupyter Notebook

4. Dataset

No external dataset was used. Student records were created manually, as suggested in the task. Each record contains a name, department, year and CGPA.

5. Data Structure Design

The main dictionary uses the student ID as the key. The value is a nested dictionary with the student's details.

```
students = {
    101: {"name": "Arun", "dept": "AI&DS", "year": 3, "cgpa": 8.4},
    102: {"name": "Priya", "dept": "CSE", "year": 2, "cgpa": 9.1},
    103: {"name": "Karthik", "dept": "ECE", "year": 4, "cgpa": 7.8},
}
```

Student ID is an integer and unique, so it works well as a key. Lookup by key takes constant time on average.

6. Implementation

6.1 Search

`search_student(sid)` uses `dict.get()`. If the ID exists, the record is printed and returned. If not, a "not found" message is shown, so the program does not crash with a KeyError.

6.2 Add

`add_student(sid, name, dept, year, cgpa)` first checks whether the ID already exists. A duplicate ID is rejected with a message, which prevents accidental overwriting of existing data. Otherwise a new record is inserted.

6.3 Update

`update_student(sid, **changes)` takes the ID and any fields to change as keyword arguments. It uses `dict.update()` on the nested record, so one or many fields can be modified in a single call. Updating a missing ID shows a message instead of an error.

7. Example Outputs

```
Found 102: {'name': 'Priya', 'dept': 'CSE', 'year': 2, 'cgpa': 9.1}
Student ID 999 not found
Student Meena added with ID 104
ID 101 already exists. Use update_student instead.
Updated 103: {'name': 'Karthik', 'dept': 'ECE', 'year': 4, 'cgpa': 8.2}
ID 555 not found. Use add_student first.
```

8. Results and Observations

- All three operations work correctly on valid input.
- Edge cases (missing ID, duplicate ID) are handled with clear messages.
- Using functions keeps the code modular and easy to extend, for example with delete or filter-by-department operations.

9. Interview Questions

What is a dictionary in Python? A mutable collection of key-value pairs. Each key maps to a value, and lookup by key is very fast.

Why are keys important in dictionaries? Keys identify and give access to values. They must be unique and hashable (for example int, str, tuple). A repeated key overwrites the earlier value.

How is a dictionary different from a list? A list is accessed by index position, while a dictionary is accessed by key. Dictionaries give faster lookups and more meaningful access, such as `students[101]` instead of `students[0]`.

10. Conclusion

The project shows how dictionaries can represent structured records and support basic database-like operations. The same idea scales to larger tasks such as handling JSON data or building lookup tables in data science projects.

11. Future Improvements

- Add delete and list-by-department operations.
- Save and load records from a JSON or CSV file.
- Add input validation for CGPA range and year.